Edge cases (a missing required argument) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangChain/Python). Read Basic first even if you already know this — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

This exercise isn't really about writing new code — it's about watching real model behavior on purpose, so you're not surprised by it later.

Give a tool a required argument (like `city: str`), then ask a question that clearly needs the tool but never gives it a city. The model has three real options here: ask you a clarifying question back, guess a city, or refuse to call the tool at all. There's no code fix needed yet — your job is to run it several times and see which one actually happens.

Things to use:

- A tool with one required argument, no default value — `city: str`, not `city: str = "London"`.
- A prompt that needs the tool but never names a city.
- Run it in a loop (5 times is enough) and print, each time, whether a tool call happened and what arguments it carried.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Reuse a tool from an earlier exercise (`get_weather(city: str)` works fine) rather than building a new one — the point here is the *prompt*, not the tool.

Run the same ambiguous prompt (something like "what's the weather like") several times, and log, for each run: did the model call the tool at all, and if so, what value did it send for `city`? A model might send an empty string, a guessed default city, or genuinely refuse the call and respond with plain text asking you for the city.

Look specifically at **`response.content` vs. `response.tool_calls`:** when the model refuses to call the tool and asks you a question instead, that question lands in `response.content`, and `response.tool_calls` will be empty — check both every time, don't assume one or the other. Three buckets are enough to classify each run: "asked a clarifying question," "called the tool with a guessed/empty value," and "called the tool with something else unexpected."

This is exactly why "checking arguments" (Core Concepts) isn't the whole story: Pydantic only checks that a `city` value *exists and is a string* — it can't tell you whether the model quietly guessed "London" as a default. That's a values problem, not a shape problem, and it's worth seeing the difference here. Write your classification function's signature before moving to Hint 2.

Difference between Basic and Intermediate: same underlying loop (ask 5 times, observe what happened) at 2 completeness levels — Basic just prints what happened, Intermediate sorts each run into named buckets (asked, guessed empty, guessed a value) so you can actually count the split afterward. Worth knowing neither version stops an empty-string guess from being accepted — a `Field(min_length=1)` constraint via `args_schema=` is the real fix, and worth reaching for once you're ready to move past observing.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define get_weather(city) as before, city has no default

register it on a model

repeat 5 times:
    ask "what's the weather like" (no city mentioned)
    if the model called the tool:
        print "called with city =", the argument it sent
    else:
        print "asked instead:", the model's text reply
```

Here is almost the whole thing:

```
# missing_argument_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_weather(city: str) -> str:
    """Get the current weather for a named city. Requires a city name."""
    return f"Sunny in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather])
```

What's missing: the loop of 5 runs, and printing what actually happened each time. Add those, then check the Solution.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
# missing_argument_practice.py
def classify_response(response) -> str:
    if not response.tool_calls:
        return "asked_or_answered_directly"
    city_sent = response.tool_calls[0]["args"].get("city")
    if not city_sent:
        return "called_with_empty_value"
    return "called_with_guessed_value"
```

Wire this into a 5-run loop yourself, printing each run's outcome (and the model's reply when it asked instead of calling).

Then fix it at the tool (Approach 2): a Pydantic model with `city: str = Field(min_length=1)`, attached with `@tool(args_schema=WeatherArgs)`. Now `get_weather.invoke({"city": ""})` raises `ValidationError` — catch it by name and return `"Error: a city name is required - ..."`. Compare against the Solution.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution